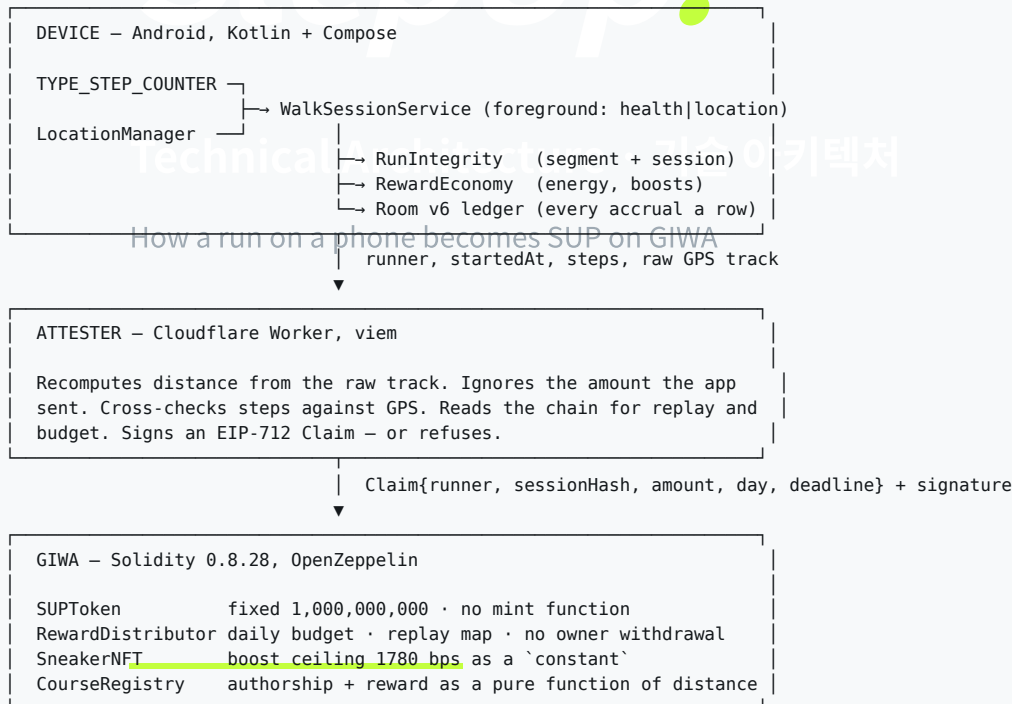


PRODUCT ARCHITECTURE & TECHNICAL IMPLEMENTATION

1. System overview

StepUp has three planes, and the split between them is the design.



App

v1.15.1 · Android · Kotlin + Jetpack Compose

The division of labour, stated once: the attester decides *who gets how much*; the contract decides *how much may exist*. Fraud detection is a moving target and belongs off chain where it can be updated. A supply cap is not, and belongs on chain where it cannot.

Attester

Cloudflare Worker · viem · EIP-712

Chain

GIWA Sepolia · chain 91342 · 4 contracts live

Repository

github.com/mycyi1994-hash/GIWASTEPN

2. Client architecture

Concern	Choice	Where
Language / UI	Kotlin 2.0.21, Jetpack Compose + Material 3, Navigation Compose	app/
SDK	minSdk 26 · targetSdk 35 · compileSdk 35 · JVM target 17	app/build.gradle.kts
Pattern	MVVM + Repository, StateFlow end to end, no LiveData Every number in this document is a constant you can open in the repository. 이 문서의 모든 숫자는 저장소에서 열어 확인할 수 있는 상수입니다.	ui/screens/*/ ViewModel.kt
DI	Manual — one object ServiceLocator, wired in Application.onCreate	core/ServiceLocator.kt
Persistence	Room 2.6.1 via KSP — strideup.db, schema v6, 12 entities, 12 DAOs	data/local/
Settings	DataStore (UserPrefs) — step baselines, locale, goals, wallet	data/prefs/UserPrefs.kt

Concern	Choice	Where
Domain	Pure Kotlin objects, no Android imports, unit-tested	<code>domain/</code>
i18n	665 strings × ko / en / zh / ja + <code>localeConfig</code> in-app override	<code>res/values*/strings.xml</code>

Why manual DI. One `object` with a dozen `lateinit` fields replaces Hilt's annotation processor, its build-time cost and its lifecycle rules. The graph is one screen long and readable top to bottom; nothing in the app needs scoped injection beyond "one instance per process."

Room entities (v6): `DailySteps` · `WalkSession` · `Reward` · `Sneaker` · `Boost` · `ClaimedEvent` · `Crew` · `CrewMembership` · `Post` · `Comment` · `Course` · `Notification`.

The `Reward` table is the ledger. Every accrual and every spend is a row, which is what makes the local balance auditable today and reconcilable against on-chain claims later. `fallbackToDestructiveMigration()` is deliberate for this milestone — while the schema is still moving, rebuilding local demo data beats crashing on a bad migration. It comes out when the schema freezes.

3. Measurement — from sensor to distance

3.1 Steps

`sensor/StepTracker.kt` wraps `TYPE_STEP_COUNTER`, which reports *cumulative steps since boot*. That raw value is useless on its own; three corrections make it a day counter:

- **Day rollover** — at a date change the current cumulative value becomes the new baseline, so today starts at zero without losing the sensor's continuity.
- **Reboot** — a cumulative value below the stored baseline means the counter reset; the baseline drops to zero and today's already-persisted steps are re-applied as an offset instead of being lost.
- **Ordering** — sensor events are funnelled through a `CONFLATED` channel into a single coroutine, so two events can never race on the `DataStore` baseline.

`hasReading` guards the initial `StateFlow` value of `0`: session accounting only trusts emissions after the first real sensor event, which is what stopped "steps walked before the app was opened" from vanishing (v1.15.1).

3.2 GPS and the run session

`service/WalkSessionService.kt` is a foreground service typed `health|location`, so a run survives screen-off and app switching.

GPS provider	<code>requestLocationUpdates(GPS_PROVIDER, 2500 ms, 6 m)</code>
Fallback	<code>NETWORK_PROVIDER, 4000 ms, 10 m</code> when GPS is disabled
Control	<code>ACTION_START</code> / <code>PAUSE</code> / <code>RESUME</code> / <code>STOP</code> intents
UI binding	<code>WalkSessionService.state: StateFlow<WalkSessionState></code> on the companion — screens observe state, never the service

Party	Party size arrives as an intent extra; the multiplier is applied at settlement
-------	--

3.3 Distance and the course map

Distance is **Haversine per segment** ($R = 6,371,000$ m), summed only over segments that pass the integrity check in § 4. Rendering normalizes the polyline with a `cos(latitude)` correction so a course drawn at Seoul's latitude is not horizontally stretched, simplifies it, and draws it on a `Canvas` over OSM tiles — start dot, finish flag, live runner dot, automatic lap split every kilometre plus a manual lap button.

4. Run integrity — proving it was a run

`domain/RunIntegrity.kt`. A step sensor cannot tell a runner from a phone rattling in a car, so GPS segment speed is judged alongside it. Two layers plus one independent axis:

Layer	Rule	Constant
Segment	Above human speed → the segment's distance is discarded entirely and never drawn	<code>MAX_SPEED_KMH = 25.0</code>
Segment (noise floor)	Under 5 m or under 1 s is not judged — punishing GPS jitter costs honest runners	<code>MIN_SEGMENT_METERS = 5.0</code> , <code>MIN_SEGMENT_SEC = 1</code>
Session	≥ 3 flagged segments <i>and</i> flagged ratio ≥ 0.5 → <code>VOID</code> , no reward	<code>MIN_FLAGS_FOR_VOID = 3</code> , <code>VOID_FLAG_RATIO = 0.5</code>
Cadence	Over 240 steps/min after a 60 s grace window → <code>VOID</code> regardless of GPS	<code>MAX_CADENCE_SPM = 240.0</code> , <code>CADENCE_GRACE_SEC = 60</code>

`verdict()` returns `CLEAN` · `FLAGGED` · `VOID` ; only `VOID` blocks accrual. 25 km/h sits above any real runner (marathon WR pace ≈ 20.9 km/h, sampled over 2.5 s windows) and below cycling and driving. Every function here is pure and covered by unit tests that run in CI.

5. Reward economy

`domain/RewardEconomy.kt` — pure functions, no Android, unit-tested.

```

earnableSteps = floor(energy × 600 / energyEfficiency)
rewardedSteps = min(walkedSteps, earnableSteps)
points        = rewardedSteps × 0.01 × sneaker × party × boost
energyUsed    = rewardedSteps × energyEfficiency / 600

```

Parameter	Value
Accrual	0.01 SUP per step, only during an active run session

Parameter	Value
Energy	1 cell = 600 rewardable steps , 10 cells base, +2 per sneaker level, refilled at midnight. At 0 energy accrual stops entirely
Sneaker boost	$\text{rarity} + \text{variant} \times 0.3\% + (\text{level} - 1) \times 0.5\% \rightarrow \text{max } +17.8\%$ (Legendary variant 1, Lv.30)
Comfort	Up to 15% less energy per step, clamped to $[0.5, 1.0]$
Party run	$1 + 0.10 \times \min(\text{size} - 1, 5) \rightarrow \times 1.5 \text{ max}$
XP booster	$\times 2$ for 24 h, costs 200 SUP
Course completion	$\text{km} \times 1.0 \text{ SUP}$, capped at 42, paid at $\geq 98\%$ coverage
Sinks	Mint $500 \cdot \text{upgrade level} \times 100 \times (1 + \text{rarity} \times 0.25)$ · boosts 50–200

The ceiling is the thesis. The strongest possible sneaker earns **+17.8%** over a free one — not $3\times$, not $10\times$. Capital cannot outrun legs, and that is enforced by a `constant` in three codebases rather than by an operator's promise. Full model and per-persona balance tables: [TOKENOMICS.md](#).

6. Attester — the off-chain judge

`attester/` — one Cloudflare Worker, `viem`, 7 passing tests, free tier covers 100k requests/day.

It does not trust the client. The app is installable, patchable and therefore forgeable; the amount it reports is not read at all. The Worker recomputes everything from the raw GPS polyline.

`POST /claim` runs, in order:

- Shape** — valid address; finite timestamps; $\text{elapsed} \geq 60 \text{ s}$; $0 < \text{steps} \leq 48,000$; track has ≥ 2 points; `endedAt` not in the future.
- Recompute** — `inspectTrack()` walks the polyline with the *same* Haversine and the *same* thresholds as the client, then `verdict()`. `VOID` $\rightarrow$ HTTP 422, no signature, nothing to submit.
- Cross-check** — $\text{steps} \times 0.762 \text{ m}$ against GPS-measured distance. Outside $[0.5\times, 2.0\times]$ one of the two was fabricated $\rightarrow$ rejected.
- Payout** — computed server-side in integer math at $1e18$ scale so no float ever reaches the chain; boost clamped to `MAX_BOOST_BPS = 1780`, party capped.
- Session hash** — `keccak256("runner|startedAt|steps|round(distanceM)")`, **rebuilt by the server**, never accepted from the client. This is the key the contract dedupes on.
- Chain state** — `sessionClaimed(hash)` $\rightarrow$ 409 if already paid; `dayRemaining(day)` $\rightarrow$ 429 if today's budget is spent (so no user ever pays gas for a claim that would revert).
- Sign** — EIP-712 `Claim(address runner, bytes32 sessionHash, uint256 amount, uint64 day, uint256 deadline)`, domain `StepUpRewards / 1 / chainId 91342 / verifyingContract`, **deadline now + 10 minutes** so a stolen signature has a short life.

The signing key lives as a Worker Secret in a wallet distinct from the deployer. Its blast radius is bounded by § 8.

7. Contracts on GIWA

Solidity **0.8.28**, OpenZeppelin, Hardhat, **43 passing tests**. All four deployed to **GIWA Sepolia (chain 91342)** on **2026-07-31**, reward pool funded with **50,000,000 SUP** — `contracts/deployments/giwaSepolia.json`.

Contract	Address	What it guarantees
SUPToken	<code>0xb052A8f6...</code> <code>A9006c1B</code>	ERC-20 + Burnable + Permit. <code>TOTAL_SUPPLY = 1,000,000,000</code> , minted once in the constructor. No mint function, no minter role — supply can only fall, through the burns that mints, upgrades and boosts perform
SneakerNFT	<code>0x8174f905...</code> <code>BabEFc960</code>	ERC-721 + Enumerable + Royalty (500 bps). <code>boost = rarityBps + variant × 30 + (level - 1) × 50</code> ; the maxed Legendary is 1780 bps, a constant . Mint burns 500 SUP from <code>msg.sender</code> and requires an EIP-712 <code>MintAuth</code> from <code>roller</code> with a per-account nonce; upgrades need no signature because cost and effect are deterministic
RewardDistributor	<code>0x9f9E87bD...</code> <code>aCFE36E1</code>	EIP-712 claims against a hard daily budget : 250,000 SUP/day, halving every 730 days, the series converging to 365,000,000. 7-day claim window, <code>sessionClaimed</code> replay map, <code>Pausable</code> . No sweep, no rescue, no owner withdrawal — SUP leaves only through <code>claim</code> , only to a runner
CourseRegistry	<code>0x6c815DF0...</code> <code>C588542</code>	Authorship of a course, on the record. <code>rewardFor</code> is a pure function of distance — <code>SUP_PER_KM = 1</code> , <code>MAX_REWARD = 42</code> , <code>MIN_DISTANCE_M = 300</code> , <code>COMPLETION_THRESHOLD_BPS = 9800</code> . The polyline stays off chain; <code>polylineHash</code> is stored so the served track can be checked

Anyone may submit a claim, and the SUP always goes to `c.runner`. That is deliberate: a runner who has never touched a chain should not need ETH to receive their first reward, so a relay can pay the gas without being able to redirect the payout.

8. Trust model — what breaks if a key leaks

Decision	Made where	Why there
Who ran, and how much they earn	Attester (off chain)	Fraud rules must change as abuse changes
How much SUP may exist	<code>RewardDistributor</code> (on chain)	A cap that can be edited is not a cap
What a sneaker earns	<code>SneakerNFT</code> constants	Earning power must be publicly verifiable, not asserted
What a course pays	<code>CourseRegistry.rewardFor</code>	A runner can compute their reward before starting

If the attester key is stolen outright: the thief can misdirect *one day's emission budget*. They cannot mint SUP, cannot exceed the daily budget, cannot claim a session twice, and cannot drain the pool — every claim is charged

against `dailyBudget(day)` and the pool has no withdrawal path. The owner rotates the key with `setAttester` and can `pause` while doing it. That bounded blast radius is precisely what makes it acceptable to keep the judge off chain.

The privileged surface is the whole list: `setAttester` , `pause` / `unpause` , `setRoller` , `setRecorder` , `setBaseURI` , `setRoyaltyReceiver` . There is no third-party-funds path anywhere in it.

9. One economy, three codebases, kept in sync by tests

The same constants exist in Kotlin, JavaScript and Solidity. If they ever diverge, the app shows one number and the chain pays another — the kind of bug a user notices first. So each side is pinned by tests:

Codebase	Holds	Tests
<code>app/src/main/java/.../domain/</code>	<code>RewardEconomy.kt</code> , <code>RunIntegrity.kt</code>	23 Android unit tests — <code>./gradlew :app:testDebugUnitTest</code>
<code>attester/src/economy.js</code>	Mirror of both, explicitly documented as a mirror	7 tests — <code>npm test</code>
<code>contracts/contracts/*.sol</code>	<code>MINT_COST</code> , <code>VARIANT_BPS</code> , <code>LEVEL_BPS</code> , <code>STEPS_PER_ENERGY</code> , <code>SUP_PER_KM</code> , <code>MAX_REWARD</code>	43 tests — <code>npx hardhat test</code> , asserting the on-chain constants equal the client's, that the boost ceiling really is 1780 bps, and that the reward pool has no owner withdrawal path

10. End-to-end: a run becomes a claim

```
1 User taps START RUN
2 WalkSessionService goes foreground (health|location); sensors attach
3 Every GPS fix → Haversine segment → RunIntegrity.isPlausible?
  └─ no → distance discarded, segment flagged, not drawn
  └─ yes → distance accumulated, track drawn, top speed updated
4 Every step tick → RewardEconomy.sessionReward(steps, energy, sneaker, party, boost)
5 User taps STOP → RunIntegrity.verdict(valid, flagged, steps, elapsed)
  └─ VOID → session saved, reward zero, user told why
  └─ else → SUP credited to the Room ledger, energy debited
6 [next milestone] POST /claim {runner, startedAt, endedAt, steps, boostBps, partySize, track}
7 Attester recomputes, cross-checks, reads the chain, signs EIP-712 or refuses
8 RewardDistributor.claim(Claim, signature)
  → signer == attester? sessionHash unused? within 7 days?
  → amount ≤ dayRemaining(day)? ≤ poolBalance()?
  → SUP transferred to the runner, session marked claimed, Claimed emitted
```

Steps 1–5 are shipped and running on the APK today. Steps 6–8 are written, tested and deployed as contracts; the wiring inside the app is milestone M3.

11. Build, release and verification

Android CI — `.github/workflows/build-apk.yml`, on every push:

```
:app:testDebugUnitTest → :app:assembleDebug → signature fingerprint check → apk-dist
```

The fingerprint step compares the APK's certificate against the committed debug keystore with `keytool`. A CI runner is a fresh machine, so an auto-generated keystore would sign every build differently and every update would fail on-device with `INSTALL_FAILED_UPDATE_INCOMPATIBLE`. A fixed keystore plus a hard check in CI means yesterday's install can always be updated in place.

Contracts — Hardhat; `scripts/deploy.js` writes `deployments/giwaSepolia.json`, and Blockscout verification is scripted three ways (`verify.js`, `standard-json.js`, `split-sources.js`) because explorers differ on how they accept multi-file sources.

Verify this document yourself:

Claim	Command
Economy and integrity constants hold	<code>./gradlew :app:testDebugUnitTest</code>
Contract constants match the client	<code>cd contracts && npm i && npx hardhat test</code>
Attester mirrors the client	<code>cd attester && npm i && npm test</code>
The app is real	Install the APK on any Android 8.0+ phone
The contracts are real	Open any address in § 7 on sepolia-explorer.giwa.io

Scale: 81 Kotlin source files · 60+ screens · 665 strings × 4 languages · Room schema v6 · 44 sneaker designs · 100 achievements · 60 runner levels · 73 tests across three codebases.

12. What is not done, stated plainly

- **The app still settles locally.** Rewards are written to the Room ledger. The app does not yet connect a wallet or call `claim` — that is milestone M3.
- **The attester is written and tested but not deployed.** `wrangler.toml` already points at the live `RewardDistributor`; publishing it is a five-minute step that has not been taken.
- **NFT metadata is a placeholder.** `baseURI` is `ipfs://REPLACE_WITH_CID/` until the 44 designs are pinned.
- **Community, ranking and course sharing are local + seeded.** There is no backend yet; that is M4.
- **No external audit.** 43 tests are not an audit, and we do not present them as one.

Milestone	Scope
M1 — done	Full client: run tracking, courses, NFTs, community, i18n, CI, installable APK

Milestone	Scope
M2 — done	Four contracts deployed to GIWA Sepolia, 50M SUP pool funded, 43 tests. Remaining: source verification on the explorer
M3	Wallet connect + on-chain claim in the app; attester published
M4	Backend for community, ranking and course sharing
M5	NFT marketplace (trade / rent), Health Connect, decentralised attestation

StepUp — 기술 아키텍처 · 한국어

폰에서 댄 러닝이 GIWA 위의 SUP가 되기까지

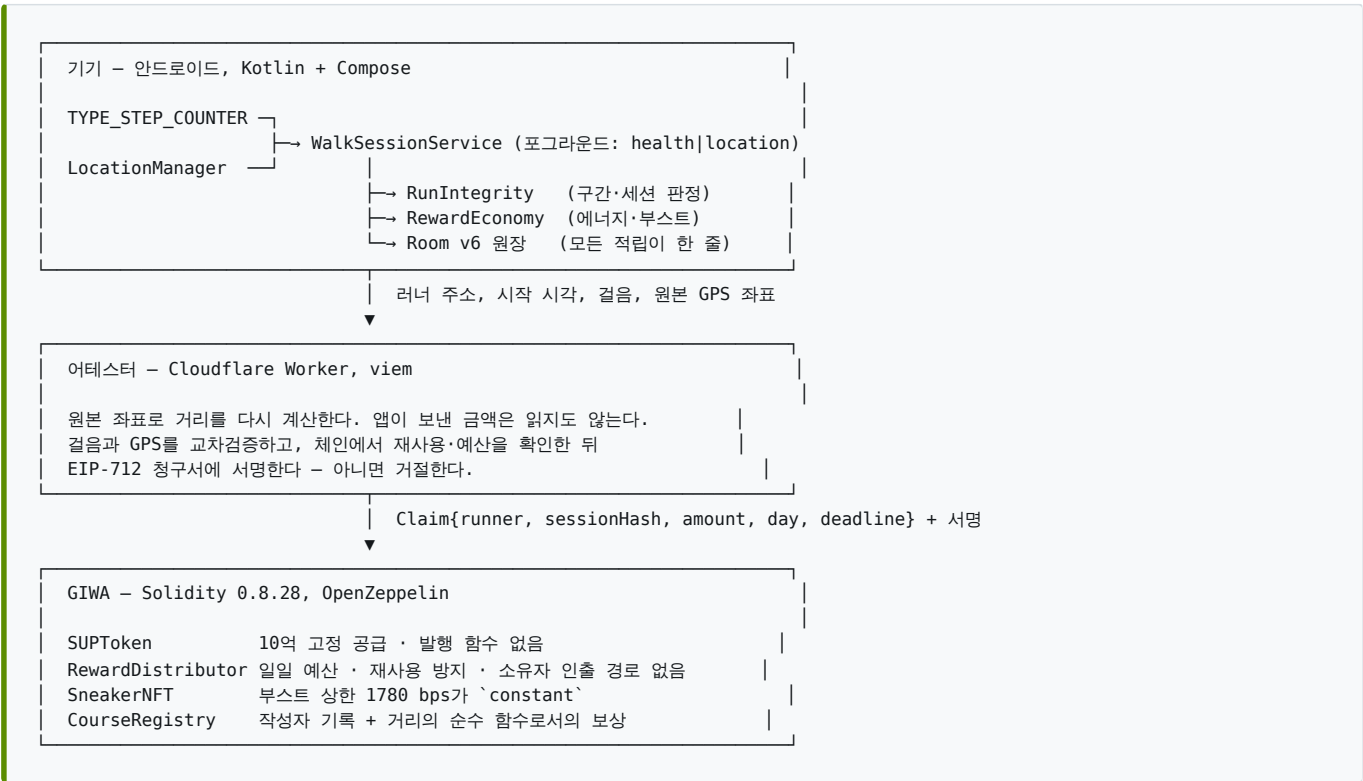
제품 아키텍처와 기술 구현 문서. 이 문서의 모든 숫자는 저장소에서 열어 확인할 수 있는 상수입니다.

앱 v1.15.1 · 안드로이드 · Kotlin + Jetpack Compose · Cloudflare Worker 어테스터 · 컨트랙트 4종 **GIWA Sepolia(체인 91342)** 배포 완료

↓ **APK** · [저장소](#) · [원페이지](#) · [토크노믹스](#) · [컨트랙트](#) · [어테스터](#)

1. 시스템 개요

StepUp은 세 개의 층으로 나뉘고, 그 나뉨 자체가 설계입니다.



역할 분담을 한 문장으로: 어테스터는 누가 얼마를 받는지를 정하고, 컨트랙트는 얼마나 존재할 수 있는지를 정합니다. 부정 탐지 규칙은 계속 바뀌어야 하므로 고칠 수 있는 오프체인에 두고, 공급 상한은 절대 바뀌면 안 되므로 고칠 수 없는 온체인에 둡니다.

2. 클라이언트 아키텍처

항목	선택	위치
언어 / UI	Kotlin 2.0.21, Jetpack Compose + Material 3, Navigation Compose	app/

항목	선택	위치
SDK	minSdk 26 · targetSdk 35 · compileSdk 35 · JVM 17	app/build.gradle.kts
패턴	MVVM + Repository, 전 구간 StateFlow, LiveData 미사용	ui/screens/*/...ViewModel.kt
DI	수동 — object ServiceLocator 하나를 Application.onCreate 에서 초기화	core/ServiceLocator.kt
저장소	Room 2.6.1 (KSP) — strideup.db, 스키마 v6, 엔티티 12개, DAO 12개	data/local/
설정	DataStore(UserPrefs) — 걸음 기준점, 언어, 목표, 지갑	data/prefs/UserPrefs.kt
도메인	순수 Kotlin, 안드로이드 의존 없음, 단위 테스트 대상	domain/
다국어	문자열 665개 × ko / en / zh / ja + localeConfig 앱 내 언어 변경	res/values*/strings.xml

왜 수동 DI인가. lateinit 필드 열몇 개짜리 object 하나가 Hilt의 애너테이션 프로세서와 빌드 비용, 생명주기 규칙을 대신합니다. 그래프가 한 화면에 들어오고 위에서 아래로 읽힙니다. "프로세스당 하나" 이상의 스코프가 필요한 곳이 앱에 없습니다.

Room 엔티티(v6): DailySteps · WalkSession · Reward · Sneaker · Boost · ClaimedEvent · Crew · CrewMembership · Post · Comment · Course · Notification .

Reward 테이블이 **원장**입니다. 모든 적립과 사용이 한 줄로 남기 때문에 지금은 로컬 잔액을 감사할 수 있고, 나중에는 온체인 청구와 대사(reconcile)할 수 있습니다. fallbackToDestructiveMigration() 은 이번 마일스톤에서의 **의도된 선택**입니다. 스키마가 아직 움직이는 동안에는 마이그레이션 실패로 앱이 죽는 것보다 로컬 데모 데이터를 다시 만드는 편이 낫습니다. 스키마가 굳으면 뱉니다.

3. 측정 — 센서에서 거리까지

3.1 걸음

sensor/StepTracker.kt 는 TYPE_STEP_COUNTER 를 감쌉니다. 이 센서는 **부팅 이후 누적 걸음**을 주기 때문에 그대로는 쓸 수 없고, 세 가지 보정을 거쳐야 "오늘 걸음"이 됩니다.

- **자정 롤오버** — 날짜가 바뀌면 현재 누적값을 새 기준으로 저장해, 센서의 연속성을 잃지 않고 오늘을 0부터 셉니다.
- **재부팅** — 누적값이 저장된 기준점보다 작으면 센서가 초기화된 것이므로 기준점을 0으로 내리고, 오늘 이미 저장된 걸음은 오프셋으로 되살립니다.
- **순서 보장** — 센서 이벤트를 CONFLATED 채널로 모아 코루틴 하나에서 처리해, 두 이벤트가 DataStore 기준점을 두고 경합하지 않게 합니다.

hasReading 플래그는 StateFlow 초기값 0 을 막습니다. 첫 실제 센서 이벤트 이후의 방출부터만 세션 집계에 쓰이며, **앱을 열기 전에 걸은 걸음이 사라지던 문제**(v1.15.1)를 고친 것이 이 플래그입니다.

3.2 GPS와 러닝 세션

service/WalkSessionService.kt 는 health|location 타입의 포그라운드 서비스라, 화면을 끄거나 앱을 전환해도 러닝이 살아 있습니다.

GPS	requestLocationUpdates(GPS_PROVIDER, 2500 ms, 6 m)
-----	--

폴백	GPS가 꺼져 있으면 <code>NETWORK_PROVIDER</code> , 4000 ms, 10 m
제어	<code>ACTION_START</code> / <code>PAUSE</code> / <code>RESUME</code> / <code>STOP</code> 인텐트
UI 연결	companion의 <code>state: StateFlow<WalkSessionState></code> — 화면은 서비스가 아니라 상태를 구독합니다
파티런	인원은 인텐트 엑스트라로 전달되고, 배율은 정산 시점에 공급됩니다

3.3 거리와 코스맵

거리는 **구간별 하버사인**($R = 6,371,000$ m)이며, § 4의 판정을 통과한 구간만 누적합니다. 렌더링은 `cos(위도)` 보정으로 폴리라인을 정규화해 서울 위도에서 그린 코스가 가로로 늘어나지 않게 하고, 경로를 숨은 뒤 OSM 타일 위 `Canvas` 에 그립니다 — 시작 점, 도착 깃발, 실시간 러너 점, 1 km 자동 랩과 수동 랩 버튼.

4. 러닝 판정 — "정말 뛰었는가"

`domain/RunIntegrity.kt`. 걸음 센서만으로는 달리는 사람과 차 안에서 흔들리는 폰을 구분할 수 없으므로 GPS 구간 속도를 함께 봅니다. 두 층 + 독립된 한 축입니다.

층	규칙	상수
구간	사람 속도를 넘으면 그 구간 거리는 아예 누적하지 않고 지도에도 그리지 않음	<code>MAX_SPEED_KMH = 25.0</code>
구간(노이즈 하한)	5 m 미만·1초 미만은 판정하지 않음 — GPS 튜를 부정으로 몰면 정상 러너가 손해	<code>MIN_SEGMENT_METERS = 5.0</code> , <code>MIN_SEGMENT_SEC = 1</code>
세션	뒀 구간이 3회 이상 그리고 비율 0.5 이상이면 <code>VOID</code> , 적립 없음	<code>MIN_FLAGS_FOR_VOID = 3</code> , <code>VOID_FLAG_RATIO = 0.5</code>
케이던스	60초 유예 뒤 분당 240보를 넘으면 GPS와 무관하게 <code>VOID</code>	<code>MAX_CADENCE_SPM = 240.0</code> , <code>CADENCE_GRACE_SEC = 60</code>

`verdict()` 는 `CLEAN` · `FLAGGED` · `VOID` 를 돌려주고, 적립을 막는 것은 `VOID` 뿐입니다. 25 km/h는 어떤 러너도 넘기 어렵고 (마라톤 세계기록 평균 ≈ 20.9 km/h, 2.5초 간격 평균 속도) 자전거·자동차는 확실히 걸러지는 지점입니다. 여기 모든 함수가 순수 함수이고 CI의 단위 테스트가 지킵니다.

5. 리워드 이코노미

`domain/RewardEconomy.kt` — 순수 함수, 안드로이드 의존 없음, 단위 테스트 대상.

```
적립 가능 걸음 = floor(에너지 × 600 / 에너지효율)
보상 걸음      = min(실제 걸음, 적립 가능 걸음)
포인트        = 보상 걸음 × 0.01 × 신발 × 파티 × 부스트
에너지 소모    = 보상 걸음 × 에너지효율 / 600
```

항목	값
적립	1보당 0.01 SUP, 러닝 세션 중에만
에너지	1칸 = 적립 가능 600보 , 기본 10칸, 신발 레벨당 +2칸, 자정 리필. 0이면 적립 완전 중단
신발 부스트	등급 + 변형 × 0.3% + (레벨 - 1) × 0.5% → 최대 +17.8% (전설 변형1, Lv.30)
착화감	걸음당 에너지 최대 15% 절감 , [0.5, 1.0] 로 클램프
파티런	$1 + 0.10 \times \min(\text{인원} - 1, 5)$ → 최대 ×1.5
XP 부스터	24시간 ×2, 가격 200 SUP
코스 완주	$\text{km} \times 1.0 \text{ SUP}$, 최대 42, 98% 이상 주파 시 지급
소각처	민팅 500 · 강화 레벨 × 100 × (1 + 등급 × 0.25) · 부스트 50~200

상한이 곧 주장입니다. 최강 신발이 무료 신발보다 **+17.8%** 더 법니다 — 3배도, 10배도 아닙니다. 자본이 다리를 이길 수 없고, 그것을 운영자의 약속이 아니라 세 코드베이스의 `constant` 가 강제합니다. 전체 모델과 페르소나별 밸런스 표: [TOKENOMICS.md](#).

6. 어테스터 — 오프체인 심판

`attester/` — Cloudflare Worker 하나, `viem`, 테스트 7개 통과, 무료 티어로 하루 10만 요청.

클라이언트를 믿지 않습니다. 앱은 고쳐서 다시 설치할 수 있으니 앱이 말한 금액은 읽지도 않습니다. Worker가 원본 GPS 폴리라인으로 전부 다시 계산합니다.

`POST /claim` 이 하는 일, 순서대로:

- 형식** — 유효한 주소, 유한한 시각, 경과 $\geq 60\text{초}$, $0 < \text{걸음} \leq 48,000$, 좌표 2점 이상, `endedAt` 이 미래가 아닐 것.
- 재계산** — `inspectTrack()` 이 클라이언트와 **같은 하버사인, 같은 임계값**으로 폴리라인을 훑고 `verdict()` 를 냅니다. VOID 면 HTTP 422, 서명 없음.
- 교차검증** — 걸음 × 0.762 m 와 GPS 거리 비교. $[0.5\times, 2.0\times]$ 를 벗어나면 둘 중 하나가 조작된 것이므로 거절.
- 지급액** — 서버가 $1e18$ 스케일 정수 연산으로 산출해 부동산수점이 체인 금액에 끼어들지 않게 하고, 부스트는 `MAX_BOOST_BPS = 1780` 으로, 파티는 상한으로 클램프.
- 세션 해시** — `keccak256("runner|startedAt|steps|round(distanceM)")` 를 **서버가 다시 만듭니다**. 클라이언트가 보낸 해시는 받지 않습니다. 컨트랙트가 재사용을 막는 열쇠가 이 값입니다.
- 체인 상태** — `sessionClaimed(hash)` 면 409, `dayRemaining(day)` 가 모자라면
- 되돌려질 트랜잭션에 유저가 가스를 쓰지 않게 합니다.
- 서명** — `EIP-712 Claim(address runner, bytes32 sessionHash, uint256 amount, uint64 day, uint256 deadline)`, 도메인 `StepUpRewards / 1 / chainId 91342 / verifyingContract`, **유효시간 10분**으로 탈취된 서명이 오래 살지 않게 합니다.

서명 키는 배포자와 **다른 지갑**의 Worker Secret으로 보관합니다. 털렸을 때의 피해 범위는 § 8에 정확히 적었습니다.

7. GIWA 위의 컨트랙트

Solidity 0.8.28, OpenZeppelin, Hardhat, 테스트 43개 통과. 4종 모두 2026-07-31 GIWA Sepolia(체인 91342)에 배포됐고 리워드 풀에 5천만 SUP가 들어가 있습니다 — `contracts/deployments/giwaSepolia.json`.

컨트랙트	주소	무엇을 보장하나
SUPToken	0xb052A8f6... A9006c1B	ERC-20 + Burnable + Permit. TOTAL_SUPPLY = 10억 을 생성자에서 한 번만 발행. 발행 함수도, 발행 권한도 없음 — 민팅·강화·부스트의 소각으로 줄어들 수만 있습니다
SneakerNFT	0x8174f905... BabEFc960	ERC-721 + Enumerable + Royalty(500 bps). 부스트 = 등급bps + 변형 × 30 + (레벨 - 1) × 50, 전설 만렙이 1780 bps이고 constant 입니다. 민팅은 msg.sender 에게서 500 SUP를 소각하고 roller 의 EIP-712 MintAuth (계정별 nonce)를 요구합니다. 강화는 서명이 필요 없습니다 — 비용과 효과가 결정적이니까요
RewardDistributor	0x9f9E87bD... aCFE36E1	하드 일일 예산 아래의 EIP-712 청구: 하루 25만 SUP, 730일마다 반감, 급수는 3억 6500만으로 수렴. 청구 기한 7일, sessionClaimed 재사용 방지, Pausable . 스왑도, 레스큐도, 소유자 인출도 없음 — SUP는 claim 으로 러너에게만 나갑니다
CourseRegistry	0x6c815DF0... C588542	코스 작성자를 기록에 남깁니다. rewardFor 는 거리의 순수 함수 — SUP_PER_KM = 1, MAX_REWARD = 42, MIN_DISTANCE_M = 300, COMPLETION_THRESHOLD_BPS = 9800. 폴리라인은 오프체인에 두고 polylineHash 만 저장해, 앱이 준 경로를 대조할 수 있게 합니다

청구는 누구나 제출할 수 있고, SUP는 언제나 c.runner 에게 갑니다. 의도된 설계입니다. 체인을 처음 만지는 러너가 첫 보상을 받으려고 ETH부터 구할 필요는 없어야 하므로, 릴레이어가 가스를 대신 낼 수 있되 수취인을 바꿀 수는 없게 했습니다.

8. 신뢰 모델 — 키가 털리면 무엇이 무너지나

결정	어디서	왜 거기인가
누가 뛰었고 얼마를 버나	어테스터(오프체인)	부정 수법이 바뀌면 규칙도 바뀌어야 함
SUP가 얼마나 존재할 수 있나	RewardDistributor (온체인)	고칠 수 있는 상한은 상한이 아님
신발이 얼마를 버나	SneakerNFT 상수	적립력은 주장이 아니라 공개 검증 대상이어야 함
코스가 얼마를 주나	CourseRegistry.rewardFor	러너가 뛰기 전에 직접 계산할 수 있어야 함

어테스터 키가 통째로 털려도: 흥친 쪽이 할 수 있는 건 하루치 매출 예산을 엉뚱한 곳으로 보내는 것뿐입니다. SUP를 발행할 수 없고, 일일 예산을 넘길 수 없고, 같은 세션을 두 번 청구할 수 없고, 풀을 비울 수도 없습니다 — 모든 청구가 dailyBudget(day) 에서 차감되고 풀에는 인출 경로가 없기 때문입니다. 소유자는 setAttester 로 키를 교체하고 그동안 pause 할 수 있습니다. 피해 범위가 이렇게 묶여 있다는 것이, 심판을 오프체인에 두어도 되는 이유입니다.

권한 있는 함수는 이게 전부입니다: setAttester, pause / unpause, setRoller, setRecorder, setBaseURI, setRoyaltyReceiver. 남의 자금에 닿는 경로는 하나도 없습니다.

9. 하나의 이코노미, 세 코드베이스 — 테스트가 일치를 지킨다

같은 상수가 Kotlin·JavaScript·Solidity에 각각 있습니다. 한쪽만 어긋나면 앱이 보여준 금액과 체인이 지급한 금액이 달라지고, 그건 유저가 가장 먼저 알아채는 종류의 버그입니다. 그래서 세 면 모두 테스트로 못을 박았습니다.

코드베이스	담긴 것	테스트
app/src/main/java/.../domain/	RewardEconomy.kt , RunIntegrity.kt	안드로이드 단위 테스트 23개 — <code>./gradlew :app:testDebugUnitTest</code>
attester/src/economy.js	위 둘의 거울(문서에 거울이라고 명시)	테스트 7개 — <code>npm test</code>
contracts/contracts/*.sol	MINT_COST , VARIANT_BPS , LEVEL_BPS , STEPS_PER_ENERGY , SUP_PER_KM , MAX_REWARD	테스트 43개 — <code>npx hardhat test</code> . 온체인 상수가 앱과 같은지, 부스트 천장이 정말 1780 bps인지, 리워드 풀에 소유자 인출 경로가 없는지까지 검사

10. 전 구간 — 러닝이 청구가 되기까지

```
1 유저가 START RUN
2 WalkSessionService 포그라운드 진입(health|location), 센서 연결
3 GPS 갱신마다 → 하버사인 구간 → RunIntegrity.isPlausible?
    └─ 아니오 → 거리 버림, 구간 플래그, 지도에 안 그림
    └─ 예 → 거리 누적, 경로 렌더, 최고속도 갱신
4 걸음 갱신마다 → RewardEconomy.sessionReward(걸음, 에너지, 신발, 파티, 부스트)
5 유저가 STOP → RunIntegrity.verdict(정상, 플래그, 걸음, 경과)
    └─ VOID → 세션은 저장, 보상 0, 이유를 유저에게 표시
    └─ 그 외 → Room 원장에 SUP 적립, 에너지 차감
6 [다음 마일스톤] POST /claim {runner, startedAt, endedAt, steps, boostBps, partySize, track}
7 어테스터가 재계산·교차검증·체인 조회 후 EIP-712 서명 또는 거절
8 RewardDistributor.claim(Claim, signature)
    → 서명자가 어테스터인가? 세션 해시가 미사용인가? 7일 이내인가?
    → 금액 ≤ dayRemaining(day)? ≤ poolBalance()?
    → 러너에게 SUP 전송, 세션 청구 완료 표시, Claimed 이벤트
```

1~5번은 지금 APK에서 실제로 돌아갑니다. 6~8번은 작성·테스트가 끝났고 컨트랙트도 배포됐지만, **앱 안에서의 연결이 M3입니다**.

11. 빌드·배포·검증

안드로이드 CI — `.github/workflows/build-apk.yml`, 푸시마다:

```
:app:testDebugUnitTest → :app:assembleDebug → 서명 지문 검증 → apk-dist
```

지문 단계는 `keytool` 로 APK 인증서를 저장소에 고정해 둔 debug 키스토어와 비교합니다. CI 러너는 매번 새 머신이라 자동 생성 키스토어를 쓰면 빌드마다 서명이 달라지고, 기기에서 업데이트 설치가 `INSTALL_FAILED_UPDATE_INCOMPATIBLE` 로 거부됩니다. 고정 키스토어 + CI의 강제 검사로 **어제 설치한 앱 위에 항상 덮어쓸 수 있게** 했습니다.

컨트랙트 — Hardhat. `scripts/deploy.js` 가 `deployments/giwaSepolia.json` 을 쓰고, Blockscout 검증은 세 가지 경로로 스크립트화돼 있습니다(`verify.js` , `standard-json.js` , `split-sources.js`) — 탐색기마다 다중 파일 소스를 받는 방식이 다르기 때문입니다.

이 문서를 직접 검증하는 법:

주장	명령
이코노미·판정 상수가 맞다	<code>./gradlew :app:testDebugUnitTest</code>
온체인 상수가 앱과 같다	<code>cd contracts && npm i && npx hardhat test</code>
어테스터가 앱의 거울이다	<code>cd attester && npm i && npm test</code>
앱이 실재한다	APK를 Android 8.0+ 기기에 설치
컨트랙트가 실재한다	§ 7의 주소를 <code>sepolia-explorer.giwa.io</code> 에서 열기

규모: Kotlin 81개 파일 · 60개 이상 화면 · 문자열 665개 × 4개 언어 · Room 스키마 v6 · 스니커즈 44종 · 업적 100종 · 러너 레벨 60단계 · 세 코드베이스 합 테스트 73개.

12. 안 된 것도 그대로

- 앱은 아직 로컬에서 정산합니다. 보상은 Room 원장에 기록됩니다. 지갑을 연결하지도, `claim` 을 호출하지도 않습니다 — M3입니다.
- 어테스터는 작성·테스트를 마쳤지만 배포 전입니다. `wrangler.toml` 은 이미 실제 `RewardDistributor` 주소를 가리키고 있고, 배포 자체는 5분짜리 단계인데 아직 하지 않았습니다.
- NFT 메타데이터가 자리표시자입니다. 44종을 IPFS에 고정하기 전까지 `baseURI` 는 `ipfs://REPLACE_WITH_CID/` 입니다.
- 커뮤니티·랭킹·코스 공유는 로컬 + 시드 데이터입니다. 백엔드가 아직 없고, M4입니다.
- 외부 감사를 받지 않았습니다. 테스트 43개는 감사가 아니고, 감사라고 말하지 않습니다.

마일스톤	범위
M1 — 완료	전체 클라이언트: 러닝 측정, 코스, NFT, 커뮤니티, 다국어, CI, 설치 가능한 APK
M2 — 완료	컨트랙트 4종 GIWA Sepolia 배포, 5천만 SUP 풀 적립, 테스트 43개. 남은 것: 탐색기 소스 검증
M3	앱 내 지갑 연결 + 온체인 청구, 어테스터 배포
M4	커뮤니티·랭킹·코스 공유 백엔드
M5	NFT 마켓(거래/임대), Health Connect, 어테스테이션 탈중앙화